

# Metrics Instrumentation

---

**Series:** OTEL | **Notebook:** 5 of 8 | **Created:** January 2026 | **Last Updated:** 01/28/2026

## Creating Custom Metrics with OpenTelemetry

---

OpenTelemetry metrics provide quantitative measurements of your application's behavior over time. This notebook covers metric types, instrumentation patterns, and integration with Dynatrace.

---

## Table of Contents

---

1. Metric Types
  2. Instrument Selection
  3. Creating Metrics
  4. Metric Attributes
  5. Aggregation and Views
  6. Prometheus Integration
  7. Best Practices
  8. Next Steps
-

# Prerequisites

| Requirement | Details                         |
|-------------|---------------------------------|
| Knowledge   | OTEL-01 and OTEL-03             |
| Environment | Collector with metrics pipeline |

## 1. Metric Types

### OTel Metric Instruments

| Instrument    | Aggregation         | Use Case            | Example            |
|---------------|---------------------|---------------------|--------------------|
| Counter       | Sum (monotonic)     | Cumulative events   | Total requests     |
| UpDownCounter | Sum (non-monotonic) | Current values      | Active connections |
| Histogram     | Distribution        | Value ranges        | Response time      |
| Gauge         | Last value          | Current measurement | Temperature        |

### Sync vs Async

| Type                      | When Recorded     | Use Case             |
|---------------------------|-------------------|----------------------|
| Synchronous               | On each operation | Request handling     |
| Asynchronous (Observable) | On collection     | System stats, gauges |

## 2. Instrument Selection

---

### Decision Guide

| Question                   | If Yes    | If No                      |
|----------------------------|-----------|----------------------------|
| Value only goes up?        | Counter   | UpDownCounter or Histogram |
| Measuring duration/size?   | Histogram | Counter or UpDownCounter   |
| Current state?             | Gauge     | Counter or Histogram       |
| Recording in request path? | Sync      | Async (Observable)         |

### Common Patterns

| Metric           | Instrument      | Why                      |
|------------------|-----------------|--------------------------|
| Request count    | Counter         | Monotonically increasing |
| Request duration | Histogram       | Distribution needed      |
| Active requests  | UpDownCounter   | Goes up and down         |
| Queue size       | ObservableGauge | Current state            |
| CPU usage        | ObservableGauge | Periodic measurement     |

## 3. Creating Metrics

---

### Python Examples

```
from opentelemetry import metrics
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
from opentelemetry.exporter.otlp.proto.grpc.metric_exporter import
OTLPMetricExporter

# Setup
exporter = OTLPMetricExporter(endpoint="http://collector:4317")
reader = PeriodicExportingMetricReader(exporter,
export_interval_millis=60000)
provider = MeterProvider(metric_readers=[reader])
metrics.set_meter_provider(provider)

# Get meter
meter = metrics.get_meter(__name__)

# Counter
request_counter = meter.create_counter(
    name="http.server.request.count",
    description="Total HTTP requests",
    unit="1"
)

# Histogram
request_duration = meter.create_histogram(
    name="http.server.request.duration",
    description="HTTP request duration",
    unit="ms"
)

# UpDownCounter
active_requests = meter.create_up_down_counter(
    name="http.server.active_requests",
    description="Active HTTP requests",
    unit="1"
)
```

## Using Metrics

```
import time

def handle_request(request):
    # Increment active requests
    active_requests.add(1, {"method": request.method})
    start = time.time()

    try:
        response = process_request(request)
        status = response.status_code
    except Exception:
        status = 500
        raise
    finally:
        # Record duration
        duration_ms = (time.time() - start) * 1000
        request_duration.record(duration_ms, {
            "method": request.method,
            "status": str(status)
        })

        # Increment counter
        request_counter.add(1, {
            "method": request.method,
            "status": str(status)
        })

        # Decrement active requests
        active_requests.add(-1, {"method": request.method})

    return response
```

## Observable (Async) Metrics

```
import psutil

# Observable gauge for CPU
def get_cpu_usage(options):
    yield metrics.Observation(psutil.cpu_percent(), {})

cpu_gauge = meter.create_observable_gauge(
    name="system.cpu.usage",
    callbacks=[get_cpu_usage],
    description="CPU usage percentage",
    unit="%"
)

# Observable counter for network bytes
last_bytes = {"sent": 0, "recv": 0}

def get_network_io(options):
    net = psutil.net_io_counters()
    yield metrics.Observation(net.bytes_sent, {"direction": "sent"})
    yield metrics.Observation(net.bytes_recv, {"direction": "recv"})

network_counter = meter.create_observable_counter(
    name="system.network.io",
    callbacks=[get_network_io],
    description="Network I/O bytes",
    unit="By"
)
```

## 4. Metric Attributes

### Common Attributes

| Attribute                     | Example                              | Use                    |
|-------------------------------|--------------------------------------|------------------------|
| <code>http.method</code>      | <code>GET</code> , <code>POST</code> | HTTP method breakdown  |
| <code>http.status_code</code> | <code>200</code> , <code>500</code>  | Response status        |
| <code>service.name</code>     | <code>checkout-api</code>            | Service identification |
| <code>environment</code>      | <code>production</code>              | Environment            |

## Cardinality Considerations

| High Cardinality (Avoid) | Low Cardinality (Good) |
|--------------------------|------------------------|
| User ID                  | Region                 |
| Request ID               | HTTP method            |
| Timestamp                | Status code class      |
| Full URL                 | Route template         |

## Attribute Best Practices

```
# Good - Low cardinality
request_counter.add(1, {
    "method": "GET",
    "route": "/api/users/{id}", # Template, not actual ID
    "status": "2xx" # Status class
})

# Bad - High cardinality
request_counter.add(1, {
    "user_id": "user-12345", # Millions of values
    "url": "/api/users/12345", # Every request different
    "timestamp": "2026-01-26T10:00:00" # Every second different
})
```

## 5. Aggregation and Views

### Histogram Buckets

Configure explicit bucket boundaries:

```

from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.view import View,
ExplicitBucketHistogramAggregation

# Custom histogram buckets for response times
duration_view = View(
    instrument_name="http.server.request.duration",
    aggregation=ExplicitBucketHistogramAggregation(
        boundaries=[5, 10, 25, 50, 100, 250, 500, 1000, 2500, 5000, 10000]
    )
)

provider = MeterProvider(
    metric_readers=[reader],
    views=[duration_view]
)

```

## Drop Metrics via Views

```

from opentelemetry.sdk.metrics.view import DropAggregation

# Drop noisy metrics
drop_view = View(
    instrument_name="internal.debug.*",
    aggregation=DropAggregation()
)

```

## 6. Prometheus Integration

---

### Scraping Prometheus Metrics

Collector configuration:



```

receivers:
  prometheus:
    config:
      scrape_configs:
        - job_name: 'my-app'
          scrape_interval: 15s
          static_configs:
            - targets: ['app:8080']
          metrics_path: /metrics

service:
  pipelines:
    metrics:
      receivers: [prometheus, otlp]
      processors: [batch]
      exporters: [otlphttp]

```

## Exposing OTel Metrics as Prometheus

```

from opentelemetry.exporter.prometheus import PrometheusMetricReader
from prometheus_client import start_http_server

# Start Prometheus endpoint
start_http_server(8080)

# Use Prometheus reader
reader = PrometheusMetricReader()
provider = MeterProvider(metric_readers=[reader])

```

```

// Query OTel metrics in Dynatrace
timeseries avg(http.server.request.duration), by:{http.method}
| limit 10

```

```

// Request rate by status
timeseries rate = sum(http.server.request.count), by:{http.status_code}
| limit 10

```

## 7. Best Practices

---

### Naming Conventions

| Pattern                                                      | Example                                                  | Description         |
|--------------------------------------------------------------|----------------------------------------------------------|---------------------|
| <code>&lt;domain&gt;.&lt;component&gt;.&lt;metric&gt;</code> | <code>http.server.request.count</code>                   | Hierarchical naming |
| Use lowercase                                                | <code>request_count</code> not <code>RequestCount</code> | Consistency         |
| Include units                                                | <code>duration_ms</code> , <code>size_bytes</code>       | Clarity             |

### Performance Tips

| Tip                             | Reason                     |
|---------------------------------|----------------------------|
| Limit attribute cardinality     | Memory, query performance  |
| Use appropriate export interval | Balance freshness vs. load |
| Batch metrics                   | Reduce network calls       |
| Use observable for system stats | Avoid polling in hot path  |

### What to Measure

| Category        | Metrics                                         |
|-----------------|-------------------------------------------------|
| <b>RED</b>      | Rate, Errors, Duration (for services)           |
| <b>USE</b>      | Utilization, Saturation, Errors (for resources) |
| <b>Business</b> | Revenue, conversions, user actions              |

## Summary

---

In this notebook, you learned:

- Metric types: Counter, UpDownCounter, Histogram, Gauge
  - Sync vs async instruments
  - Creating and recording metrics
  - Attribute best practices and cardinality
  - Views and aggregation configuration
  - Prometheus integration patterns
- 

## References

---

- [OTel Metrics Specification](#)
  - [OTel Python Metrics](#)
  - [Dynatrace Metrics Ingest](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*